COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	10/16/26	**DATE SUBMITTED:**	10/23/26
NAME:	Garcia, Dennis James T. Hare, Andrei Angelo D.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 12.1 Algorithmic Strategies

1. PROCEDURE OUTPUT

ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)

The Problem: MINIMUM STEPS TO ONE

On a positive integer, you can perform any one of the following 3 steps.

1. Subtract 1 from it. (n = n - 1)
2. If it's divisible by 2, divide by 2. (If n % 2 == 0, then n = n / 2)
3. If it's divisible by 3, divide by 3. (If n % 3 == 0, then n = n / 3).

Now the question is, given a positive integer n, find the minimum number of steps that takes n to 1.

Examples:

1. For n = 1, output: 0
2. For n = 4, output: 2 (4 /2 = 2 /2 = 1)
3. For n = 7, output: 3 (7 -1 = 6 /3 = 2 /2 = 1)

One can think of greedily choosing the step, which makes n as low as possible and continue the same, till it reaches 1. If you observe carefully, the greedy strategy doesn't work here.

Eg: Given n = 10 , Greedy --> 10 /2 = 5 -1 = 4 /2 = 2 /2 = 1 (4 steps).
But the optimal way is --> 10 -1 = 9 /3 = 3 /3 = 1 (3 steps).

It all starts with recursion.

$$F(n) = 1 + min\{ F(n-1) , F(n/2) , F(n/3) \}\ if\ (n > 1)\ ,\ else\ 0\ (\ i.e.\ ,\ F(1) = 0\ )\ .$$

Now that we have our recurrence equation, we can right away start coding the recursion. Wait. Does it have over-lapping subproblems? YES. Is the optimal solution to a given input depends on the optimal solution of its subproblems? Yes... Bingo! It is dynamic programming. So, we just store the solutions to the subproblems we solve and use them later, as in memoization, or we start from the bottom and move up till the given n, as in dynamic programming. As it's the very first problem we are looking at here, let's see both the codes:

Memoization

```
C/C++
int memo[n+1]; // we will initialize the elements to -1 ( -1 means, not solved
it yet )
int getMinSteps ( int n )
{
if ( n == 1 ) return 0; // base case
if( memo[n] != -1 ) return memo[n]; // we have solved it already :)
int r = 1 + getMinSteps( n - 1 ); // '-1' step . 'r' will contain the optimal
answer finally
if( n%2 == 0 ) r = min( r , 1 + getMinSteps( n / 2 ) ) ; // '/2' step
if( n%3 == 0 ) r = min( r , 1 + getMinSteps( n / 3 ) ) ; // '/3' step
memo[n] = r ; // save the result. If you forget this step, then its same as
plain recursion.
return r;
}
```

Run this code and show the output in table 12-2

Bottom-Up DP

```
C/C++
int getMinSteps ( int n )
{
int dp[n+1], i;
dp[1] = 0; // trivial case
for( i = 2 ; i < = n ; i ++ )
{
dp[i] = 1 + dp[i-1];
if(i%2==0) dp[i] = min( dp[i] , 1+ dp[i/2] );
if(i%3==0) dp[i] = min( dp[i] , 1+ dp[i/3] );
}
return dp[n];
}
```

Run this code and show the output in table 12-3.

Output

Strategy	Algorithm	Analysis
Recursion	Recursive Sequential Search	The function calls itself to check the next element until the target is found or the end of the array is reached.
Brute Force	Linear Search	The algorithm checks each element one by one

		until it finds the target or finishes checking the array.
Backtracking	Depth-First Search (DFS)	The algorithm explores one path at a time and returns to a previous point when it needs to try another possible path.
Greedy	Greedy Best-First Search	Search The algorithm chooses the next step based on which option appears closest to the goal according to its evaluation rule.
Divide-and-Conquer	Merge Sort	The algorithm divides the array into smaller parts, sorts each part, and combines them to produce the final sorted array.

Table 12-1. Algorithmic Strategies and Examples

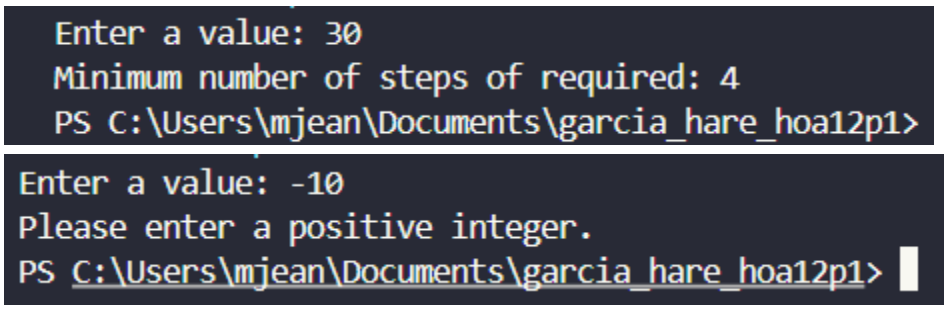
Screenshot	
Analysis	➢ Garcia: The memoization implementation uses recursion to find the minimum number of steps needed to reduce a positive integer to 1. It checks the possible operations and selects the one that requires the fewest steps. The program produced the correct result for the input of 30, which required 4 steps to reach 1. It also rejected the input of -10 because the value must be a positive integer. This shows that the program can calculate the minimum steps and validate the input before processing it. ➢ Hare: Based on our testing, the memoization program successfully calculated the minimum number of steps for the given input. It also handled invalid input by displaying a message when a negative number was entered. The program worked as expected for the tested values.

Table 12-2. Memoization Implementation

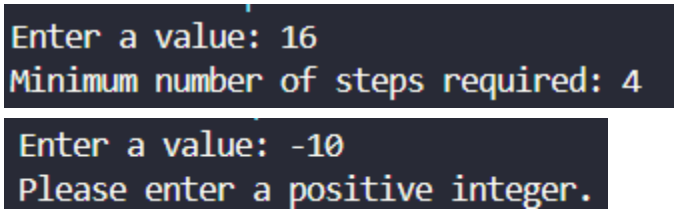
Screenshot	
Analysis	➢ Garcia: The bottom-up implementation determines the minimum steps by calculating the smaller values first before proceeding to the given number. It does not use recursive calls, which makes the process different from the previous implementation. The program returned the expected result for the input of 16 and showed that only 4 steps are needed to reach 1. When -10 was entered, the program rejected the value instead of proceeding with the calculation. This shows that the program can calculate the minimum steps and check whether the input is valid. ➢ Hare Based on the output, the bottom-up program successfully calculated the minimum number of steps for the input value of 16, which required 4 steps to reach 1. The program also handled the negative input correctly by displaying a message asking for a positive integer.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B

Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies

Problem Title: Count the number of paths in a matrix with a given cost to reach the destination cell

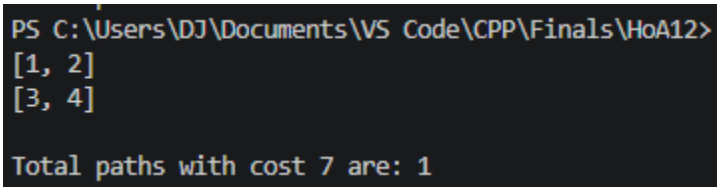
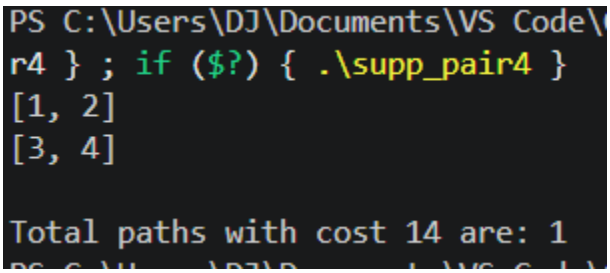
Description: Given an M × N integer matrix where each cell has a non-negative cost associated with it, count the number of paths to reach the last cell (M-1, N-1) of the matrix from its first cell (0, 0) such that the path has given cost. We can only move one unit right or one unit down from any cell, i.e., from cell (i, j), we can move to (i, j+1) or (i+1, j). For example, consider the following 4 × 4 matrix where cost is 25. Two paths are having a cost of 25. A sample path has been given when the cost is 25.

4	7	1	6
6	7	3	9
3	8	1	2
7	1	7	3

The idea is to use recursion. The problem has optimal substructure. That means the problem can be broken down into smaller, simple “subproblems”, which can further be divided into yet simpler, smaller subproblems until the solution becomes trivial.

Deliverables:	

Pseudocode (+ Solving Problem by Hand)	Psuedocode:
	None
	`Algorithm pathmemo`
	`(matrix, M, N, targetcost): [row][col][cost]`
	`    memo = M x N x (targetcost plus 1)`
	
	`    return solve(M minus 1, N minus 1, cost, matrix, memo)`
	
	`function solve(row, col, cost, matrix, memo):`
	
	`    if cost less than 0:`
	`        return 0`
	
	`    if row equates to 0 and col equates to 0:`
	`        return (matrix[0][0] equates to cost) ? 1 : 0`
	
	`    if memo[row][col][cost] is not -1:`
	`        return memo[row][col][cost]`
	
	`    remaining_cost = cost - matrix[row][col]`
	`    pathtotal = 0`
	
	`    if row greater than 0:`
	`        paths += solve(row - 1, col, remaining_cost, matrix, memo)`
	`    if col greater than 0:`
	`        paths += solve(row, col - 1, remaining_cost, matrix, memo)`

Working C++ Code	
Screenshots of demonstration	 
Analysis of working code	➢ Garcia: Based on the output and testing of the program, the code uses recursion to check the possible paths in the matrix based on the given cost. For the implementation, memoization was also used to save the results of previous calculations and avoid repeating the same process. The program checks the possible paths by moving through the matrix and subtracting the value of each cell from the target cost until it reaches the starting cell and that helps the program count the paths that match the required cost. ➢ Hare: The program was able to count the possible paths based on the given cost and display the result. It uses recursion to check the possible ways to reach the destination by moving right or down. I also learned that saving previous results helps reduce repeated calculations when solving the same problem.

```cpp
#include <iostream>
#include <vector>

int pathmemo(int m, int n, int cost, const std::vector<std::vector<int>>& mat, std::vector<std::vector<std::vector<int>>>& memo){

    if(cost < 0){
        return 0;
    }


    if(m == 0 && n == 0){
        return (mat[0][0] == cost) ? 1 : 0;
    }


    if(memo[m][n][cost] != -1){
        return memo[m][n][cost];
    }

    int paths = 0;
    int remaining_cost = cost - mat[m][n];


    if (m == 0){
        paths = pathmemo(0, n - 1, remaining_cost, mat, memo);
    }else if (n == 0){
        paths = pathmemo(m - 1, 0, remaining_cost, mat, memo);
    }else{
        paths = pathmemo(m - 1, n, remaining_cost, mat, memo) +
                pathmemo(m, n - 1, remaining_cost, mat, memo);
    }

    return memo[m][n][cost] = paths;
}
```

```
PS C:\Users\DJ\Documents\VS Code\CPP\Finals\HoA12>
[1, 2]
[3, 4]

Total paths with cost 7 are: 1
```

```
PS C:\Users\DJ\Documents\VS Code\
r4 } ; if ($?) { .\supp_pair4 }
[1, 2]
[3, 4]

Total paths with cost 14 are: 1
```

Conclusion	
Summary of the lessons learned	➢ Garcia: The lessons are mostly about recursions and memoizations. Reusing data/results that came from a function call and recursion which a function call, calls itself. ➢ Hare: I learned the importance of understanding different algorithmic strategies and how they can be used to solve programming problems. I also learned how recursion and dynamic programming can help solve problems by breaking them down into smaller parts.
Analysis of the procedure	➢ Garcia: I was presented with different algorithm strategies which show their respective known algorithms. Bottom-Up DP approach basically solves subproblems or smaller problems leading to a final solution. Memoization by my understanding is that it stores data that will be reused if the same input was done again. ➢ Hare: I observed that memoization and bottom-up dynamic programming can both find the minimum number of steps needed to reach 1. The difference is that memoization uses recursion and saves previous results, while bottom-up uses a loop to calculate the answers starting from smaller values.
Analysis of the supplementary activity	➢ Garcia: Recursion and memoization was used to calculate the paths for the matrix. Consisting of an algorithm with if statements comparing the cost and matrix sizes m and n for a function call which is reused due to recursion. Memoization uses stored data which helps determine whether that path was already taken or present with the same result ➢ Hare: I learned that recursion can be used to count the possible paths in a matrix based on the given cost. The program checks the possible paths by moving right or down until it reaches the destination. Using memoization also helps avoid repeating the same calculations.
Concluding statement / Feedback: How well did you think you did in this activity? What are	➢ Garcia: Memoization and recursion seems to be reusing or storing similar data or results. Although code implementation seems to be quite complicated depending on what will be implemented.

your areas for improvement?	➢ Hare: I believe I understood the basic concept of the activity but I still need to improve my understanding of recursion and dynamic programming. I also need to see some outputs to become more familiar with how these algorithms work in actual programs.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Idea generation, explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **To explain how the path solving should work and which costs could be used. Compiling issues with a 4x4 matrix; path is always 0, therefore we choose a 2x2 matrix seemingly easier to manage.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.